Bird Wanderer Evaluation Companion

Purpose: Use this as a rehearsal and evidence guide. It tells you what to prepare, what to do on screen, what to capture and what to explain for ten substantial evaluation cases.

Revised selection: Keep the five successful workflows. Replace the earlier corrupt-file, oversized-file and impossible-date headline cases with interrupted publication, media persistence failure and privacy revocation in a chat attachment. Retain private-record protection and discovery failure. File/date checks remain supporting validation examples.

Why these failures matter: They concern lost work, duplicate records, file/database consistency, ownership boundaries, privacy after sharing and dependence on an external service. Explain the consequence being prevented, rather than simply showing an error message.

Evidence boundary: Six focused existing automated tests passed on 27 October 2026 for observation retry/persistence/privacy, media cleanup after database failure, chat persistence/privacy, attachment revocation, provider failures and observation edits. Browser recipes were checked against source code but have not been executed for this guide. Fill actual results after rehearsal.

No application changes: This companion is documentation only. Original case sheets and Run-Demo.ps1 remain unchanged. That older runner includes the old minor cases; do not label its output as the complete revised selection.

Case	Evaluation subject	Main evidence
G-P01	Discovery and linked hotspot reports	Screen and source-labelled records
G-P02	Photo processing and EXIF protection	Upload plus server/image evidence
G-P03	Private journal persistence	Owner journal and Viewer feed
G-P04	Public sharing and location protection	Two accounts and response evidence
G-P05	Account-specific bookmarks	Reload and account comparison
G-N01	Interrupted publication and safe retry	Failure, recovery and one record
G-N02	Database failure and file cleanup	Controlled automated fault
G-N03	Unauthorised private-record access	Viewer denial and Owner access
G-N04	Privacy revoked after chat sharing	Attachment becomes unavailable
G-N05	Discovery outage and recovery	Error, journal access and retry

Preparation and evidence collection

Set up once

1. From the project folder, run .\Test.ps1 and leave that terminal open. Open the frontend address printed by the launcher and use the same address throughout. Helper tests run separate temporary servers.

2. Use two demonstration accounts in separate browser profiles: Owner and Viewer. Different normal tabs usually share the same login. Keep Owner profile visibility Public for the sharing case.

3. Keep a real JPEG/PNG below 10 MB and 40 megapixels. Prefer a non-sensitive photo with camera EXIF. Missing EXIF is acceptable but cannot prove GPS removal.

4. Pick a species and hotspot actually loaded in discovery. Record their names. Demo Wetland from the automated runner is synthetic and may not appear in the normal app.

5. Use distinctive notes: EVAL PRIVATE, EVAL PUBLIC, EVAL CHAT and EVAL RETRY. Use a genuine past time. Create separate records for privacy exercises to avoid confusing captures.

6. Open browser developer tools with F12 for Network evidence. Use Windows + Shift + S for screenshots. Save captures outside source, for example Downloads\BirdWandererEvaluation.

Capture a complete story

Success evidence: Capture the action and its result after reload or from the second account. A success toast alone does not establish persistence.

Failure evidence: Capture the triggering condition, visible error or safe unavailable state, and recovery or continued authorised access. Use the screenshot names on each case page.

Lecture sheet: Copy the guide ID/title, inputs, preconditions and expected result. Enter the result you actually observed, execution date and tester. Pending browser steps remain Not run.

Privacy: Hide passwords, session cookies, authentication headers, personal addresses and exact coordinates. Do not capture .env or provider credentials.

Presentation order: G-P01, G-P02, G-N01, G-P03, G-N03, G-P04, G-N04, G-P05, G-N05, then G-N02. Run the database case beforehand and keep its console result ready.

Group roles: One member operates the app, one explains the workflow/data, one explains errors/security and one handles evidence and remaining work. Combine roles if needed.

G-P01 Discovery and linked hotspot reports

Test purpose: Shows external-data integration and the connection between catalogue species, hotspots and recent reports.

Have ready: Owner signed in; working discovery configuration; one loaded species and hotspot. Record actual names, not synthetic fixture names.

Recreate one step at a time

1. Open Explore or Search and wait for discovery loading to finish.

2. Find a loaded bird by common/scientific name and open its details.

3. Open an associated hotspot/report and show how it relates to the species.

4. Return or reload and check that the catalogue loads again.

Expected outcome: Catalogue data loads through the protected discovery route. Missing information stays unavailable. A recent report is source data, not proof that the bird is present now.

Screenshots to capture

P01-A.png: Explore/Search with source, region and retrieval status. Paste your capture below this caption.

P01-B.png: Species detail and associated hotspot or report. Paste your capture below this caption.

Evidence limits: GET /api/discovery/catalogue supplies catalogue records. One visible card does not establish that every displayed statistic is live.

Recovery or reset: No record needs deletion. If discovery fails, demonstrate G-N05 and report the state honestly.

Say while presenting: Our discovery service organises external species, hotspot and recent-report data so users can move from finding a bird to exploring a relevant location.

Your rehearsal: Tester _____ Date _____ Actual result _____ Status: Not run / Pass / Fail. Record mismatches before presenting.

G-P02 Photo processing and EXIF protection

Test purpose: Shows upload processing, metadata extraction and protection of embedded photo information.

Have ready: Owner; valid JPEG/PNG under 10 MB and 40 megapixels; camera EXIF if available. Use a safe photo.

Recreate one step at a time

1. Open Log Observation and choose Add photograph.

2. Select the valid file and wait for processing.

3. Show the preview, Source EXIF read or EXIF unavailable status, and actual camera fields.

4. Keep the upload staged for G-N01, or complete a separate observation for later cases.

Expected outcome: Upload returns a media reference. Available EXIF is displayed. The server serves a processed JPEG with embedded metadata removed; the original file is not retained.

Screenshots to capture

P02-A.png: Uploaded preview and processing/EXIF status. Paste your capture below this caption.

P02-B.png: Camera fields and image-processing disclosure. Paste your capture below this caption.

Evidence limits: The screen shows upload/metadata feedback. EXIF removal needs inspection of the served image or the existing test; hidden fields alone do not prove it.

Recovery or reset: Leaving an unfinished form attempts to discard its staged photo. Use a fresh photo if restarting.

Say while presenting: We read available camera metadata and create a processed image for sharing. EXIF describes the photograph; it is not automatic bird recognition.

Your rehearsal: Tester _____ Date _____ Actual result _____ Status: Not run / Pass / Fail. Record mismatches before presenting.

G-P03 Private observation and journal persistence

Test purpose: Connects photo, species, hotspot, notes and visibility to persistent records and profile counts.

Have ready: Owner/Viewer sessions; fresh photo; catalogue species/hotspot; past date; EVAL PRIVATE notes.

Recreate one step at a time

1. Complete Log Observation with manually selected species, hotspot, date/time and notes.

2. Choose Private and submit.

3. Open Journal as Owner, find EVAL PRIVATE and reload to confirm persistence.

4. Check Profile for the count change. As Viewer, check that this private post is absent from Community.

Expected outcome: Owner retrieves the record after reload. Counts change appropriately. Viewer feed excludes the private record.

Screenshots to capture

P03-A.png: Completed Private form with EVAL PRIVATE. Paste your capture below this caption.

P03-B.png: Owner journal after reload. Paste your capture below this caption.

P03-C.png: Viewer feed comparison; pair with direct denial in G-N03. Paste your capture below this caption.

Evidence limits: Feed absence alone is weak because filters/pagination can hide posts. G-N03 adds direct denial. Compare counts before/after; existing accounts need not total one.

Recovery or reset: Keep this record private for G-N03. Capture private evidence before reusing it.

Say while presenting: An observation links the photo to selected field details. Private visibility controls retrieval as well as what appears in the feed.

Your rehearsal: Tester _____ Date _____ Actual result _____ Status: Not run / Pass / Fail. Record mismatches before presenting.

G-P04 Public sharing without exact coordinates

Test purpose: Shows deliberate sharing while separating hotspot-level location from private photo coordinates.

Have ready: Separate Owner/Viewer profiles; EVAL PUBLIC record; Owner profile permits sharing; safe photo.

Recreate one step at a time

1. Create EVAL PUBLIC as Public, or edit it through Journal > Edit.

2. Find it in Community as Owner and copy its share link.

3. Open the link as Viewer and show the same post and photograph.

4. For response evidence, inspect GET /api/observations/{id} as Viewer: privateLocation must be absent. Hide authentication headers.

Expected outcome: Viewer reads the permitted post/photo. A hotspot name can be shown. Exact EXIF coordinates are excluded as privateLocation from the shared record.

Screenshots to capture

P04-A.png: Owner Public setting for EVAL PUBLIC. Paste your capture below this caption.

P04-B.png: Viewer reading the same post. Paste your capture below this caption.

P04-C.png: Optional redacted shared response without privateLocation. Paste your capture below this caption.

Evidence limits: A place name alone neither proves nor disproves exact-coordinate leakage. Use response/image evidence. This is not verified species-sensitivity classification.

Recovery or reset: Keep a separate public record for G-N04; its visibility will change.

Say while presenting: Visibility decides who can read the observation. Location protection decides which location details the shared response contains. Public does not mean publishing original photo GPS.

Your rehearsal: Tester _____ Date _____ Actual result _____ Status: Not run / Pass / Fail. Record mismatches before presenting.

G-P05 Persistent account specific bookmarks

Test purpose: Shows stored user preferences and ownership separation beyond a temporary button change.

Have ready: Owner/Viewer sessions; catalogue species not already bookmarked by either account.

Recreate one step at a time

1. As Owner, open the species and click Bookmark species once.

2. Capture its selected state, then reload or reopen the species.

3. Confirm it remains saved for Owner.

4. Open the same species as Viewer and show its independent saved state.

Expected outcome: The saved reference persists for Owner and does not appear automatically for Viewer. Repeated saved=true API requests create one reference.

Screenshots to capture

P05-A.png: Owner bookmark after reload. Paste your capture below this caption.

P05-B.png: Viewer independent state for that species. Paste your capture below this caption.

Evidence limits: Clicking twice toggles save then remove. It is not the duplicate-request test; the activity test/original runner repeats the same save request.

Recovery or reset: Unbookmark once after capture if desired. Confirm state before repeating.

Say while presenting: Bookmarks are account data, not only a visual toggle. Reload checks persistence and the second account checks separation.

Your rehearsal: Tester _____ Date _____ Actual result _____ Status: Not run / Pass / Fail. Record mismatches before presenting.

G-N01 Interrupted publication and safe retry

Test purpose: Tests connectivity loss during the main save workflow and recovery without losing the staged photo or creating an extra record.

Have ready: Owner; completed unpublished form with EVAL RETRY and valid staged photo. Keep it open during the failure; developer tools available.

Recreate one step at a time

1. Open developer tools and find Request blocking using its command menu or More tools. Browser wording varies.

2. Enable an exact publication-path pattern such as */api/observations. Do not use */api/observations*: that also blocks uploads and other routes.

3. Submit. Capture the visible error and blocked POST in Network. Do not reload or leave the form.

4. Disable request blocking and submit again from the same form. Wait for success.

5. Open Journal, find EVAL RETRY and reload. Check that exactly one matching record exists.

Expected outcome: Blocked request shows Cannot reach your field records. Check your connection and retry. Restoring connectivity allows staged work to be submitted; verify one matching record.

Screenshots to capture

N01-A.png: Completed form and exact-path block. Paste your capture below this caption.

N01-B.png: Visible save error and blocked POST. Paste your capture below this caption.

N01-C.png: One EVAL RETRY record after recovery/reload. Paste your capture below this caption.

Evidence limits: This blocks a request before server processing. It does not simulate a response lost after commit. The existing observation test separately checks that identical accepted submissions return the existing record.

Recovery or reset: Turn request blocking OFF immediately. A blanket block can hide the journal too. Record actual behavior if different.

Say while presenting: A failed request must not be presented as a saved observation. After connectivity returns, users can retry and we verify the stored result.

Your rehearsal: Tester _____ Date _____ Actual result _____ Status: Not run / Pass / Fail. Record mismatches before presenting.

G-N02 Database failure and orphan file cleanup

Test purpose: Tests consistency between the photo file and its database record when persistence fails after processing.

Have ready: Use command A in the appendix. It is an isolated automated fault demonstration, not a fault to introduce into your normal database.

Recreate one step at a time

1. Open a second terminal, change to frontend and run command A. Leave the normal app alone.

2. The existing test accepts a valid photo, then installs a rejecting trigger in its temporary database.

3. A subsequent valid upload reaches the real upload route and fails during persistence with HTTP 500.

4. The test asserts that no leftover staged file remains in its media directory.

5. Capture the passing result and, if needed, the source assertions explaining the injected fault.

Expected outcome: The API error handler maps the failure to generic 500 feedback. The existing test asserts the status and absence of orphan files. It does not exercise a browser rendering the error.

Screenshots to capture

N02-A.png: Passing fault-test name and terminal summary. Paste your capture below this caption.

N02-B.png: Optional test source: rejecting trigger, 500 assertion and empty media directory assertion. Paste your capture below this caption.

Evidence limits: API error: ERR_SQLITE_ERROR is expected from this injected fault. The test passes because failure is contained. It proves cleanup, not production disaster recovery or a same-database retry.

Recovery or reset: The fixture cleans its temporary storage. Do not add triggers, stop database services or delete normal app files.

Say while presenting: An upload spans the filesystem and database. If persistence fails, the processed file must not remain orphaned. We trigger that condition in isolation and assert cleanup.

Your rehearsal: Tester _____ Date _____ Actual result _____ Status: Not run / Pass / Fail. Record mismatches before presenting.

G-N03 Unauthorised private record access

Test purpose: Tests access control on the actual record and image, beyond hiding a feed card.

Have ready: Owner private record; separate Viewer profile; canonical post link and media URL from a safe test record.

Recreate one step at a time

1. For easy link setup, make a separate record Public, copy its Community share link and image request URL, then set it to Private in Journal > Edit.

2. As Owner, reopen it to confirm it exists and is private.

3. As Viewer, open the copied post link and capture the denied destination/error.

4. As Viewer, request the copied image URL afresh and inspect its 404 response.

5. Return to Owner and reopen the same record. It must still be readable.

Expected outcome: Viewer record access returns 404 / Observation not found. Protected image returns 404 / Photo not found. Owner retains access. Automated tests also deny another account mutation/delete.

Screenshots to capture

N03-A.png: Owner private setting and identifiable record. Paste your capture below this caption.

N03-B.png: Viewer denied post and optional 404 request. Paste your capture below this caption.

N03-C.png: Owner still reading it; optional Viewer photo 404. Paste your capture below this caption.

Evidence limits: Two tabs with the Owner cookie are not another-user testing. Do not claim delete denial was demonstrated if only reading was tried. Previously downloaded images cannot be recalled.

Recovery or reset: Keep the record private or restore intended visibility after capture. Hide session credentials.

Say while presenting: Authentication tells us who a user is. Authorisation decides whether they may read this particular record. That protection applies to its photo endpoint too.

Your rehearsal: Tester _____ Date _____ Actual result _____ Status: Not run / Pass / Fail. Record mismatches before presenting.

G-N04 Privacy revoked after sharing in chat

Test purpose: Tests whether an old message bypasses a later privacy decision across journal, messages and media.

Have ready: Owner/Viewer profiles permit contact; EVAL CHAT record Public; conversation between the two accounts.

Recreate one step at a time

1. As Owner, open Messages > Start new field chat, choose Viewer and open the conversation.

2. Use Attach field photo, choose EVAL CHAT and send. Only observations already visible to the recipient are eligible.

3. As Viewer, capture the attachment while accessible.

4. As Owner, use Journal > Edit to change EVAL CHAT to Private.

5. As Viewer, refresh the conversation or wait for foreground refresh. Capture Shared observation no longer available. A fresh old-photo request must return 404.

Expected outcome: Message history remains but the attachment becomes unavailable. Its new Viewer response omits the attachment id/imageUrl. Sending it does not permanently grant media access.

Screenshots to capture

N04-A.png: Viewer attachment visible before revocation. Paste your capture below this caption.

N04-B.png: Owner makes the same observation Private. Paste your capture below this caption.

N04-C.png: Viewer unavailable attachment after refresh; optional new photo 404. Paste your capture below this caption.

Evidence limits: Already downloaded images/screenshots cannot be recalled. This tests future requests. Command B tests follower-unfollow revocation and redaction; the screen recipe uses public-to-private through the same read policy.

Recovery or reset: Restore Public only if intended. Deleting the conversation is not a substitute for testing revocation.

Say while presenting: Privacy must remain effective after sharing. We re-evaluate attachment access on reading, so an old chat message cannot become a permanent bypass.

Your rehearsal: Tester _____ Date _____ Actual result _____ Status: Not run / Pass / Fail. Record mismatches before presenting.

G-N05 Discovery outage and recovery

Test purpose: Tests external discovery failure separately from locally stored field records.

Have ready: Owner session; journal record; developer tools; catalogue request path. Normal app works before the exercise.

Recreate one step at a time

1. Open Explore. Enable request blocking only for */api/discovery/catalogue. Leave authentication and observation requests unblocked.

2. Reload to attempt a new catalogue request. Capture the discovery error and failed request.

3. Open Journal and show stored records still load from the reachable backend.

4. Disable blocking, return to Explore and choose Try again.

5. Capture recovery. Use command C for real server/provider error mapping with injected faults.

Expected outcome: Browser blocking shows Cannot reach discovery. Check your connection and try again. A provider connection failure handled by the server instead maps to 503 / Cannot reach eBird. Please try again.

Screenshots to capture

N05-A.png: Discovery banner and blocked catalogue request. Paste your capture below this caption.

N05-B.png: Stored journal remains available. Paste your capture below this caption.

N05-C.png: Discovery recovered after unblocking and Try again. Paste your capture below this caption.

Evidence limits: Whole-browser offline also disconnects the backend and does not prove provider-only isolation. Browser blocking is an analogue; command C tests provider mapping. A cached response after recovery is acceptable.

Recovery or reset: Disable all blocking and confirm Explore and Journal work normally. Do not alter real API keys or claim eBird itself was down.

Say while presenting: Discovery depends on an external service; stored records have a different dependency. We report discovery failure explicitly, preserve local access and offer retry.

Your rehearsal: Tester _____ Date _____ Actual result _____ Status: Not run / Pass / Fail. Record mismatches before presenting.

Controlled test commands and evidence limits

Working directory: Run in a second PowerShell terminal from the project root. Selected existing tests use temporary storage and do not modify application source. Node 24 and installed dependencies are required.

```
Set-Location .\frontend

$nodeForEval = Join-Path $env:USERPROFILE '.cache/codex-runtimes/codex-primary-runtime/dependencies/node/bin/node.exe'
```

Node alternative: If Node 24 is on PATH, use node instead of & $nodeForEval below.

Command A Database failure and cleanup

```
& $nodeForEval --test --test-name-pattern='upload validates real content' server/observations.test.mjs
```

Interpretation: Look for cleanup when database persistence fails and pass 1 / fail 0. The same test also includes smaller file-validation checks. Its fault is confined to temporary storage.

Command B Chat attachment revocation

```
& $nodeForEval --test --test-name-pattern='chat attachments reuse' server/messages.test.mjs
```

Interpretation: Shares an eligible observation, revokes follower access, then checks unavailable fields and photo 404. Related to, but not the same interaction as, the public-to-private screen recipe.

Command C Provider failures and retry

```
& $nodeForEval --test --test-name-pattern='missing key, upstream errors' server/discovery.test.mjs
```

Interpretation: Tests configuration, upstream/malformed/network failures and recovery. Console evidence does not establish a live eBird outage.

Command D Observation retry and ownership

```
& $nodeForEval --test --test-name-pattern='upload, private observation' server/observations.test.mjs
```

Interpretation: Tests accepted observation retry, ownership, restart persistence and privacy. It does not simulate browser blocking or a response lost after commit.

Fresh verification: On 27 October 2026 these four test groups passed, plus persistent participant-only messaging and observation editing/privacy: six selected functions. This is not a fresh complete-suite claim.

Existing runner: ..\docs\test-cases\Run-Demo.ps1 still contains the original ten cases. It is not the revised headline selection in this companion.

Presentation script and likely questions

Opening and development status

Bird Wanderer supports bird and hotspot discovery, photograph processing, field observations, journals and controlled sharing. We selected five successful workflows and five substantial failure scenarios. We will show the condition, outcome, stored-data consequence and recovery for each.

Development and acceptance work remain ongoing. Uploads currently use manual species selection from a catalogue. Automatic photograph identification is a proposed enhancement; we should call it under development only if we can show actual model work. Upload suggestions with user confirmation would be the first target, with community-feed identification a later extension.

The point of the negative cases

The negative cases examine interrupted saving, failed database persistence, unauthorised access, privacy revoked after sharing and an external-data outage. The operation is unsuccessful by design. Its test passes when the application handles it safely and the evidence matches the expected result.

Answers to keep ready

Why a controlled database fault? It reliably reaches a dangerous internal path without damaging normal data. We show the injected condition and cleanup assertions and label this automated evidence.

Does Public expose exact GPS? A hotspot name can remain visible, but exact private coordinates are excluded from shared responses. The served image has embedded metadata removed; the screen alone does not prove that removal.

Does the species dropdown mean AI? No. It searches catalogue names and the user chooses. EXIF extraction is separate from recognising the bird.

What remains? Identification feasibility/validation if agreed in scope, external push/email acceptance where configuration is missing, physical-device checks and production release acceptance. Separate source delivery from provider/device verification.

What if the live result differs? Record the actual outcome and mark Fail or Not run appropriately. Show dated prior evidence with its limits. Do not pass off a terminal test as a browser capture.

Suggested ending: These tests demonstrate selected workflows and substantial failure handling. They establish specific current properties, while remaining development and acceptance work stays explicit.

Evidence register and lecture sheet handoff

Use this page: Track captures before evaluation. Blank entries are tasks, not passes. Insert captures after case captions or in separate slides with matching filenames.

Case	Screenshots	Actual observation	Screen status
G-P01	P01-A, P01-B	_____	Not run
G-P02	P02-A, P02-B	_____	Not run
G-P03	P03-A, P03-B, P03-C	_____	Not run
G-P04	P04-A, P04-B, P04-C	_____	Not run
G-P05	P05-A, P05-B	_____	Not run
G-N01	N01-A, N01-B, N01-C	_____	Not run
G-N02	N02-A, N02-B	_____	Not run
G-N03	N03-A, N03-B, N03-C	_____	Not run
G-N04	N04-A, N04-B, N04-C	_____	Not run
G-N05	N05-A, N05-B, N05-C	_____	Not run

Copy into the lecture format

Identity: Use G-P01 to G-P05 and G-N01 to G-N05 so they are distinct from old BW-N numbers. Project: Bird Wanderer. Module: case title. Priority: High for failure cases; Medium for bookmarks.

Preconditions and inputs: Use Have ready plus Test purpose. Record account roles, catalogue names, photo characteristics and note marker. Never submit passwords or session tokens.

Steps and outcomes: Transfer numbered steps into Test Steps and inputs into Test Data. Use Expected outcome for expected results. Fill actual results only after execution and record date/tester and Pass/Fail/Not run.

Postconditions: Record what remains: one valid observation, no orphan file, intact private record, redacted attachment or recovered catalogue. Show recovery as well as the error.

Which list to present: Use this revised register. The earlier Bird_Wanderer_Test_Cases document retains the original selection. If submitting it, replace its three retired negative cases and update affected IDs/results; do not submit conflicting lists.

Source pointers: Upload/save: LogObservationScreen.tsx and observations.mjs. Editing: JournalScreen.tsx and EditObservationSheet.tsx. Chat: FieldChatScreen.tsx and messages.mjs. Discovery: App.tsx and discovery clients/server. Fault assertions: observations.test.mjs, messages.test.mjs and discovery.test.mjs.

Scope: No fabricated screenshots are included. Paste your own dated captures. Application code, production configuration and original demonstration scripts remain unchanged.